

NAMA : SHINTA BELLA NURROHMAH

NIM : 242410101040

KELAS: PWEB A

P1. PRAKTIK INDIVIDU — Sistem Pencarian & Notifikasi (100 poin)

1. Buat section yang menampilkan data cuaca kota Surabaya menggunakan API publik <https://wttr.in/Surabaya?format=j1> (gratis, tidak perlu API key). Tampilkan: nama kota, suhu saat ini (dalam °C), dan deskripsi cuaca. Gunakan `async/await` dan tampilkan loading indicator saat data sedang diambil.

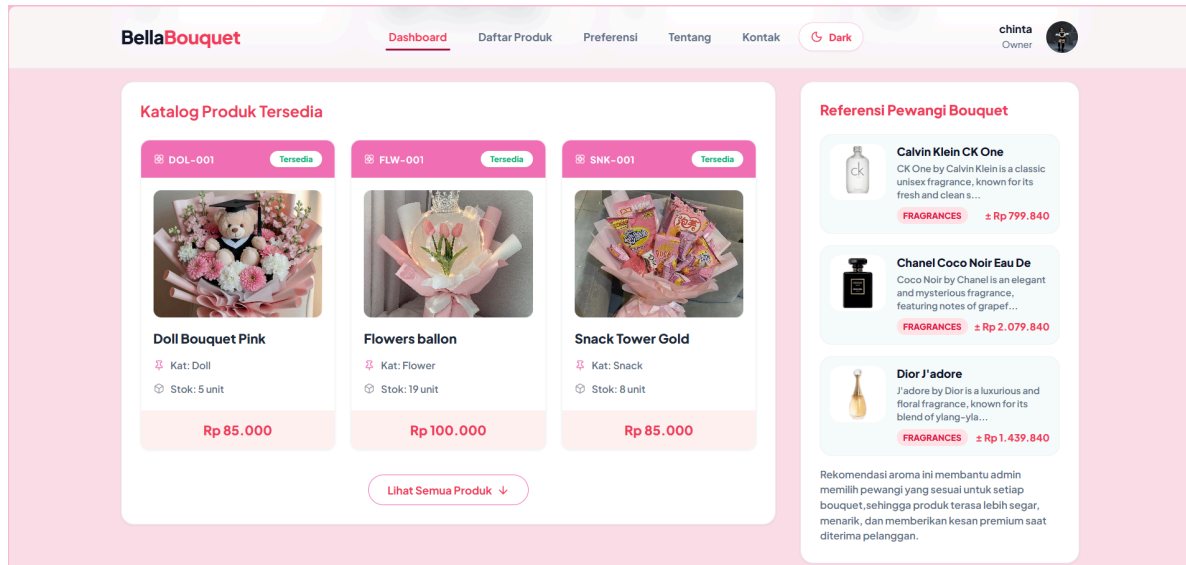
Bisa menggunakan api publik yang sesuai proyek masing masing

2. Buat live search yang memiliki form sederhana untuk mencari mahasiswa. Form dikirim via AJAX POST/ GET ke Laravel — halaman tidak boleh reload sama sekali. Setelah berhasil, catatan baru langsung muncul di daftar bawah tanpa refresh. Gunakan CSRF token yang benar.
3. Buat navbar untuk mengeset dark mode.
 - a. Tambahkan dark mode ke konfigurasi Tailwind (`darkMode: 'class'` di `tailwind.config.js`)
 - b. Buat helper function JavaScript: `setCookie`, `getCookie`, `deleteCookie`
 - c. Implementasikan toggle dark mode: klik tombol → toggle class `'dark'` di HTML → simpan ke cookie
 - d. Terapkan cookie di awal load (dalam tag script di `<head>`) untuk mencegah flash of unstyled content
 - e. Buat halaman pengaturan /preferensi dengan form: pilihan tema (`light/dark/system`), pilihan ukuran font
 - f. Simpan form ke cookie via Fetch POST ke endpoint Laravel
 - g. Laravel Controller: baca cookie dari `$request->cookie()`, kembalikan JSON konfirmasi
4. Buat section yang menghitung berapa kali pengguna mengunjungi halaman tersebut menggunakan session Laravel. Tampilkan: jumlah kunjungan, waktu kunjungan pertama, dan waktu kunjungan terakhir. Sediakan tombol "Reset Hitungan" yang menghapus data dari session dan mengulang dari awal.

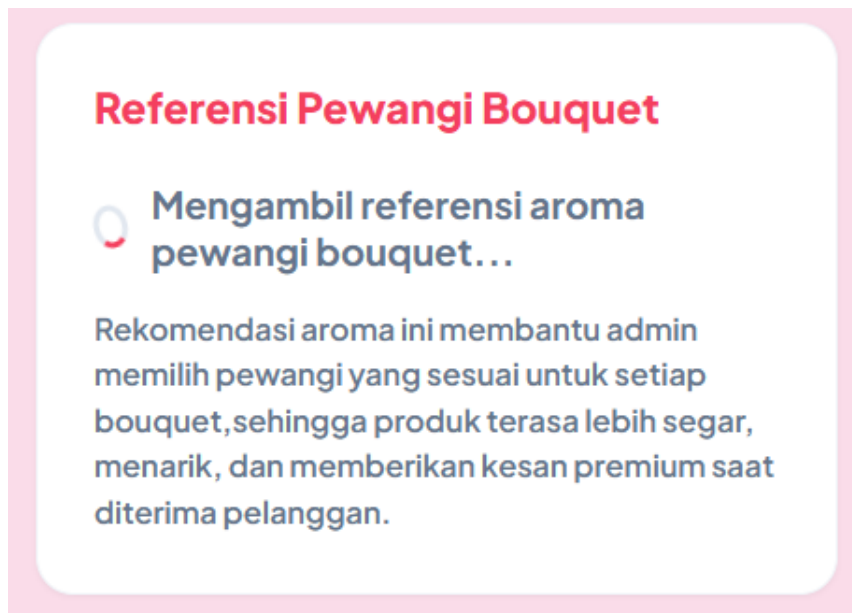
Bisa menggunakan case cart, sesuaikan dengan proyek

1. Section API Publik Menggunakan Async/Await

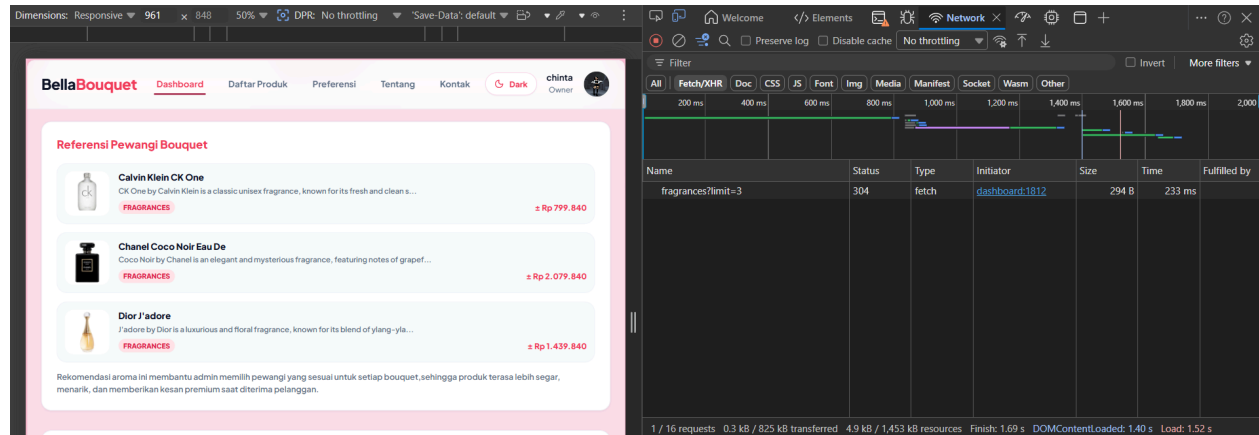
Section ini menampilkan data dari API publik dan memiliki loading indicator saat data sedang diambil. Setelah proses selesai, data akan ditampilkan dalam bentuk daftar produk/referensi aroma.



Screenshot ini membuktikan bahwa data dari API publik berhasil ditampilkan di dashboard admin.

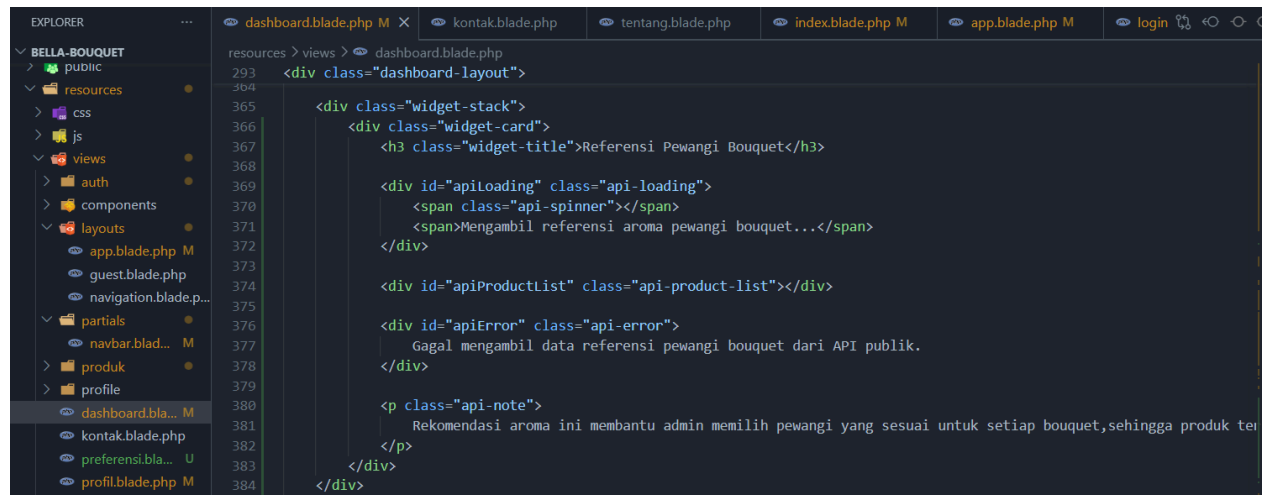


Screenshot ini membuktikan bahwa website menampilkan loading indicator saat data sedang diambil dari API.



Screenshot ini membuktikan bahwa data benar-benar diambil dari API publik.

Code :

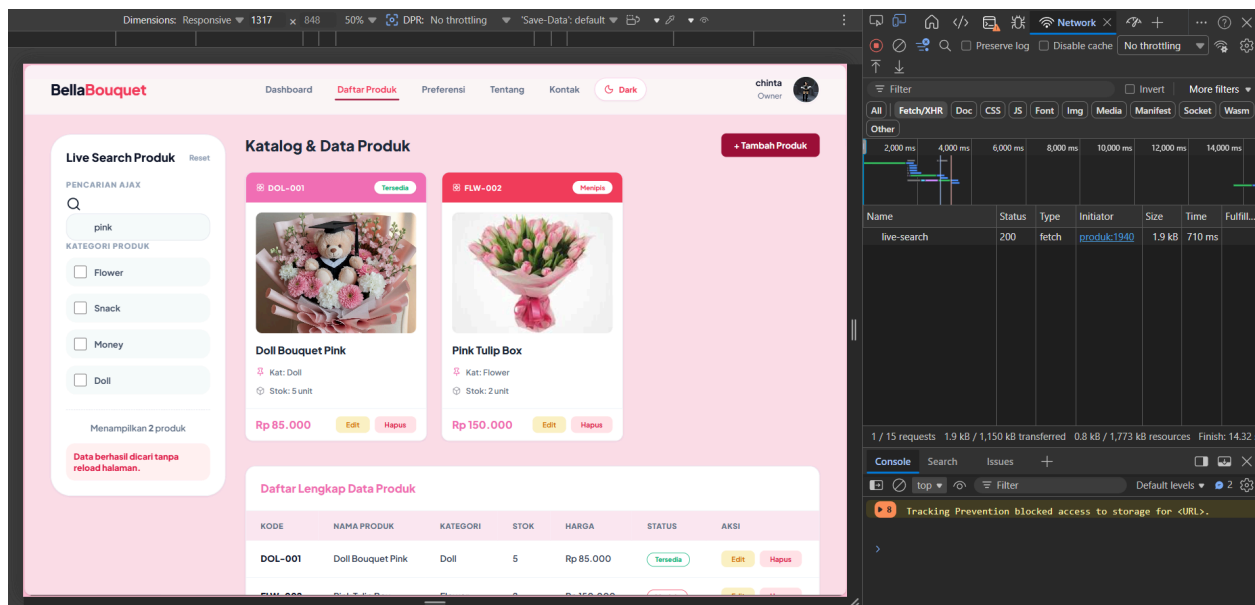


```
resources > views > dashboard.blade.php
477 <script>
478   async function getInspirasiBouquet() {
479     const loading = document.getElementById('apiLoading');
480     const list = document.getElementById('apiProductList');
481     const errorBox = document.getElementById('apiError');
482
483     try {
484       loading.style.display = 'flex';
485       list.style.display = 'none';
486       errorBox.style.display = 'none';
487
488       const response = await fetch('https://dummyjson.com/products/category/fragrances?limit=3');
489
490       if (!response.ok) {
491         throw new Error('API tidak merespon dengan benar');
492       }
493
494       const data = await response.json();
495
496       list.innerHTML = '';
497
498       data.products.forEach(function (product) {
499         const estimasiRupiah = Math.round(product.price * 16000);
500
501         list.innerHTML += `
502         <div class="api-product-card">
503           
508
509           <div class="api-product-info">
510             <div class="api-product-name">
511               ${escapeHtml(product.title)}
512             </div>
513
```

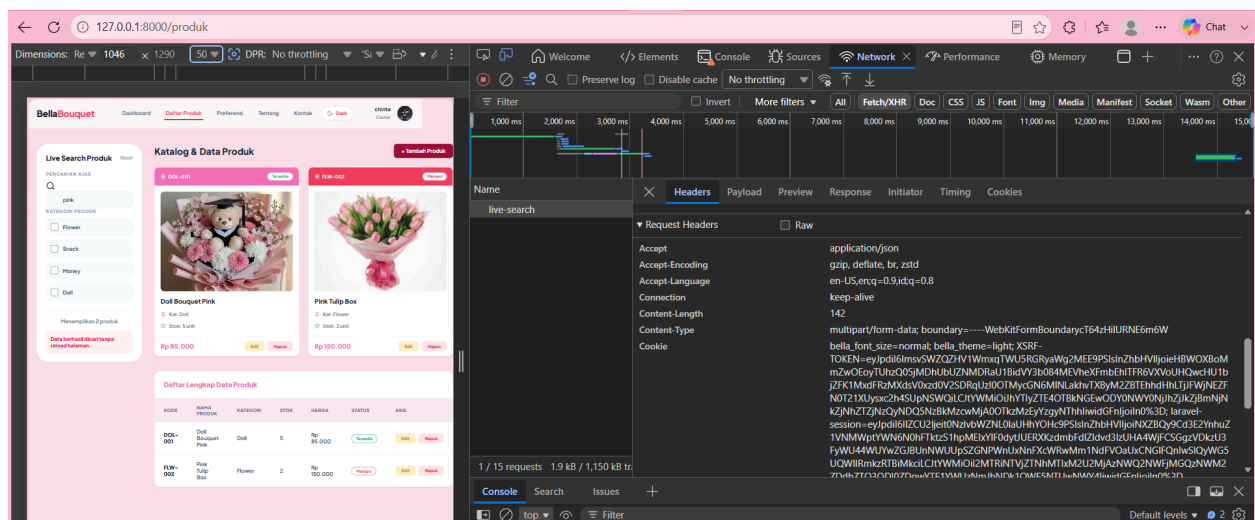
Penjelasan Singkat

Fitur ini menggunakan `fetch()` dengan `async/await` untuk mengambil data dari API publik. Saat proses pengambilan data berjalan, sistem menampilkan loading indicator. Jika data berhasil diambil, data akan muncul pada daftar referensi pewangi bouquet. Jika gagal, sistem akan menampilkan pesan error.

2. Live Search AJAX Tanpa Reload

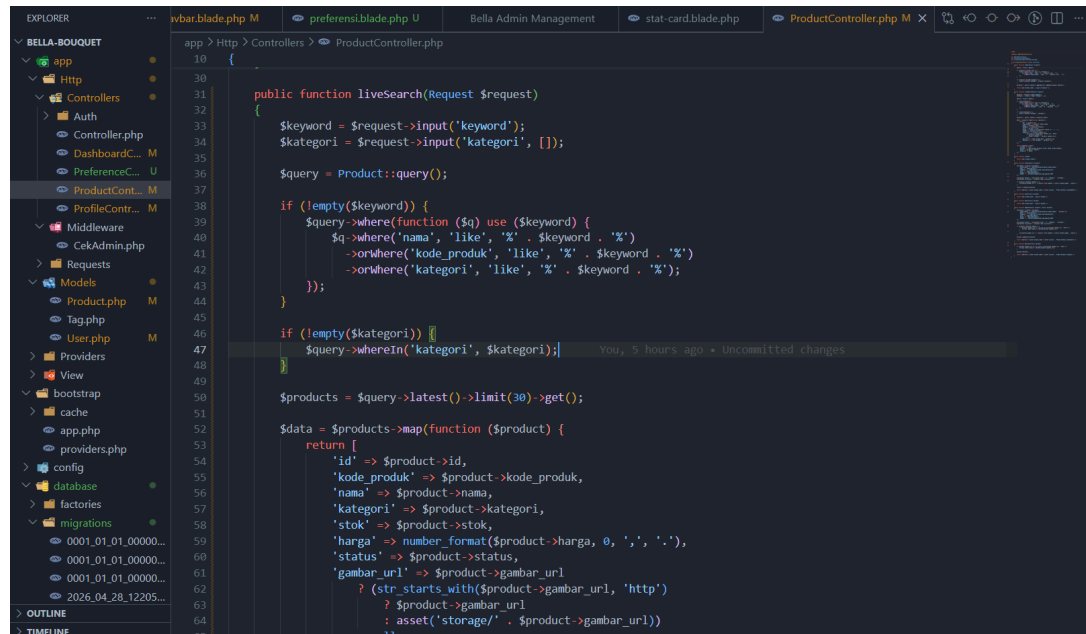


Screenshot ini membuktikan bahwa pencarian dikirim melalui AJAX ke Laravel.



Gambar di atas menunjukkan request AJAX live search ke endpoint `/produk/live-search` menggunakan method POST dan menyertakan header `X-CSRF-TOKEN`, sehingga request dapat diproses oleh Laravel tanpa error CSRF.

Code :



```
10 {
11
12     public function liveSearch(Request $request)
13     {
14         $keyword = $request->input('keyword');
15         $kategori = $request->input('kategori', []);
16
17         $query = Product::query();
18
19         if (!empty($keyword)) {
20             $query->where(function ($q) use ($keyword) {
21                 $q->where('nama', 'like', '%' . $keyword . '%')
22                     ->orWhere('kode_produk', 'like', '%' . $keyword . '%')
23                     ->orWhere('kategori', 'like', '%' . $keyword . '%');
24             });
25         }
26
27         if (!empty($kategori)) {
28             $query->whereIn('kategori', $kategori);
29         }
30
31         $products = $query->latest()->limit(30)->get();
32
33         $data = $products->map(function ($product) {
34             return [
35                 'id' => $product->id,
36                 'kode_produk' => $product->kode_produk,
37                 'nama' => $product->nama,
38                 'kategori' => $product->kategori,
39                 'stok' => $product->stok,
40                 'harga' => number_format($product->harga, 0, ',', '.'),
41                 'status' => $product->status,
42                 'gambar_url' => $product->gambar_url
43                 ? (str_starts_with($product->gambar_url, 'http')
44                     ? $product->gambar_url
45                     : asset('storage/' . $product->gambar_url))
46                 : null
47             ];
48         });
49     }
50 }
```

Method ini mengambil keyword dari request, mencari produk berdasarkan nama, kode_produk, dan kategori, lalu mengembalikan data dalam bentuk JSON. Di kode yang kamu kirim, method liveSearch() memang sudah mengembalikan response JSON dengan pesan bahwa data produk dicari tanpa reload halaman .

File Route :

```
// Live Search Produk AJAX
Route::post('/produk/live-search', [ProductController::class, 'liveSearch'])->name('produk.liveSearch');
```

File Layout untuk CSRF :

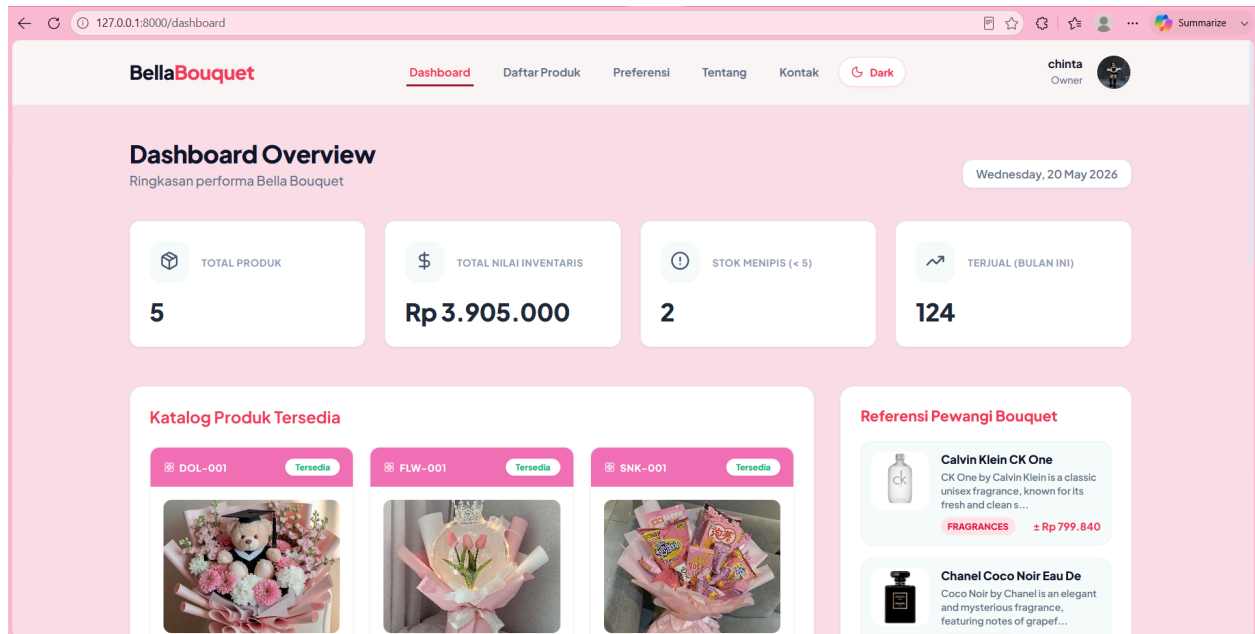


```
resources > views > layouts > app.blade.php
3 <head>
58
59 <meta name="viewport" content="width=device-width, initial-scale=1.0">
60 <meta name="csrf-token" content="{{ csrf_token() }}">
61
```

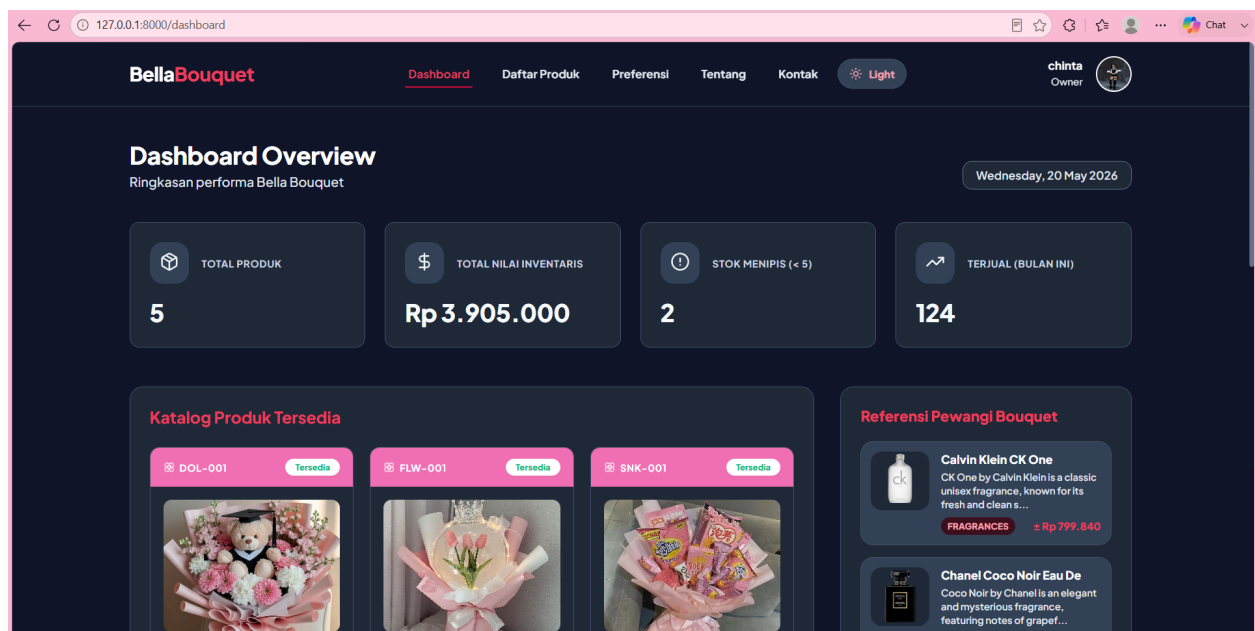
Penjelasan

Live search bekerja dengan mengirim keyword pencarian dari form ke Laravel menggunakan AJAX. Laravel memproses keyword tersebut melalui ProductController::liveSearch(), lalu mengembalikan data produk dalam bentuk JSON. Data hasil pencarian kemudian ditampilkan langsung di halaman tanpa refresh.

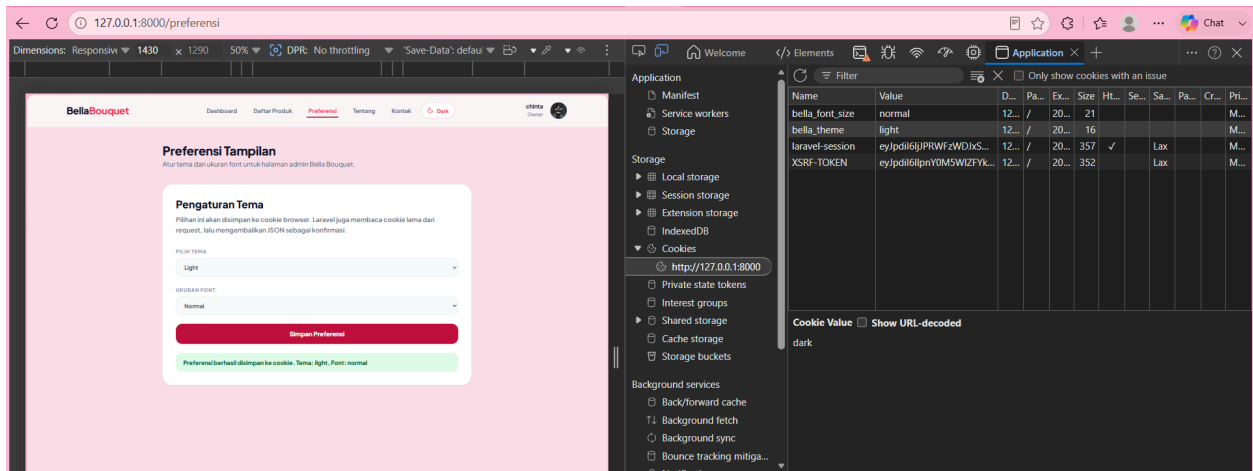
3. Navbar Dark Mode dan Halaman Preferensi



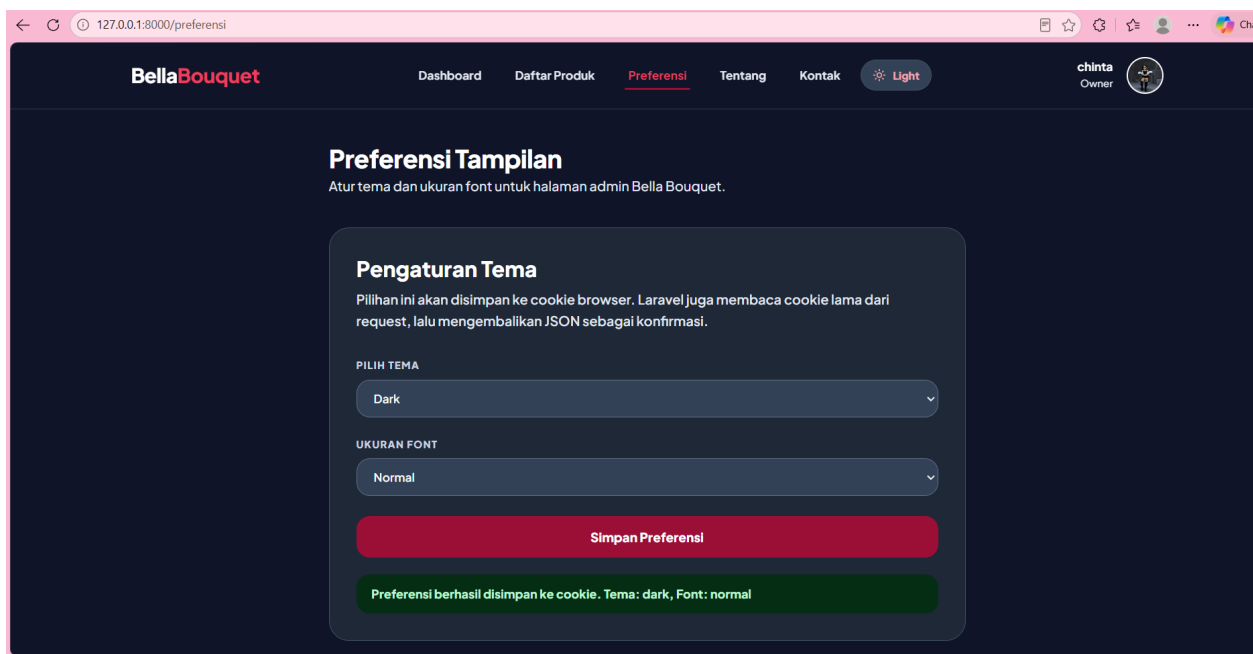
Screenshot ini menunjukkan tampilan navbar sebelum dark mode diaktifkan.



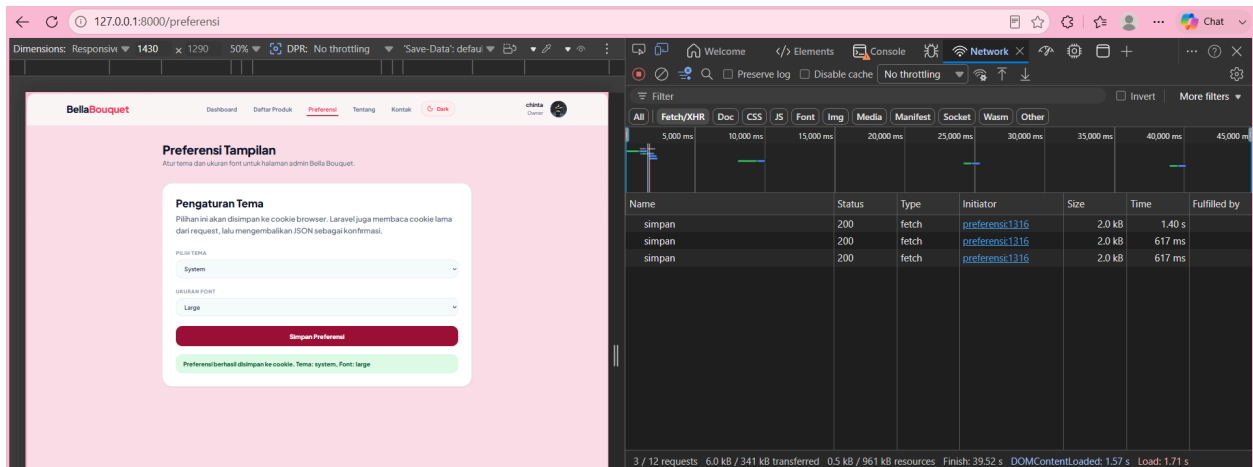
Screenshot ini menunjukkan bahwa dark mode berhasil diterapkan.



Screenshot ini membuktikan bahwa pilihan tema tersimpan di cookie.



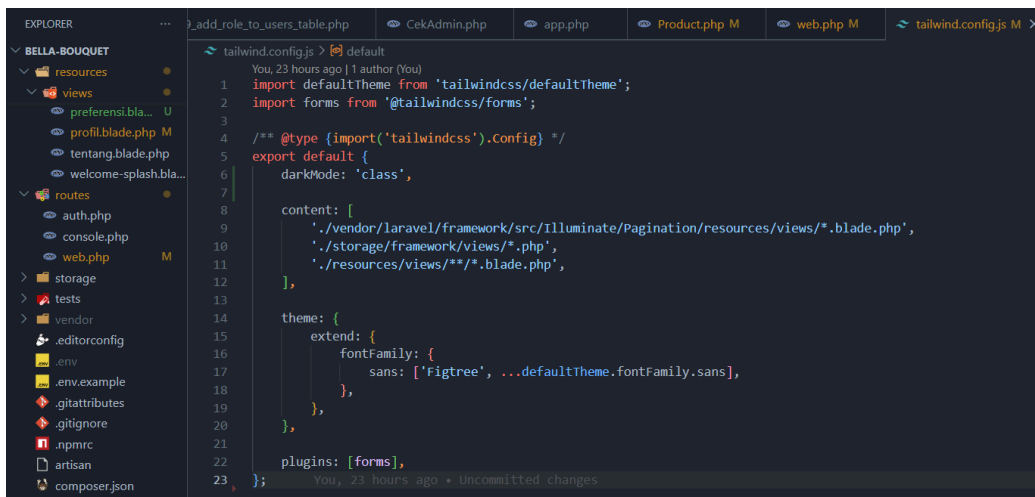
Screenshot ini membuktikan adanya halaman pengaturan preferensi.



Screenshot ini membuktikan bahwa form preferensi disimpan melalui Fetch POST ke endpoint Laravel.

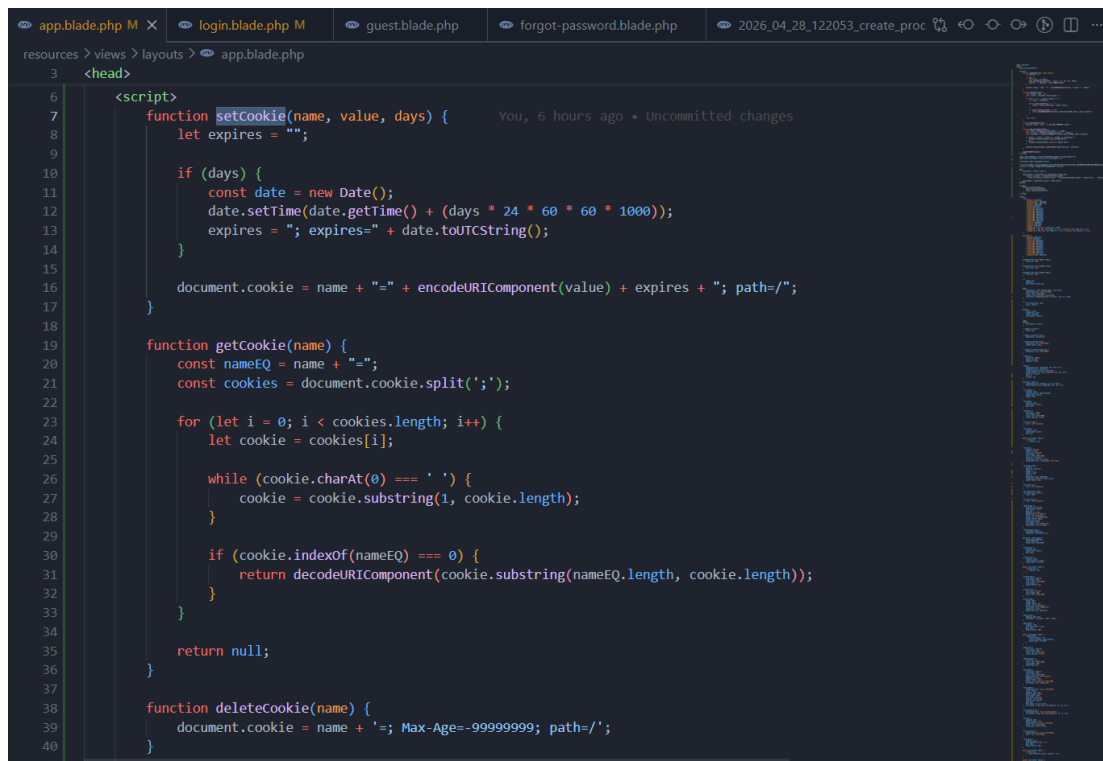
Code:

a. Konfigurasi Tailwind



Bagian ini digunakan agar dark mode bekerja berdasarkan class dark, bukan berdasarkan sistem browser saja.

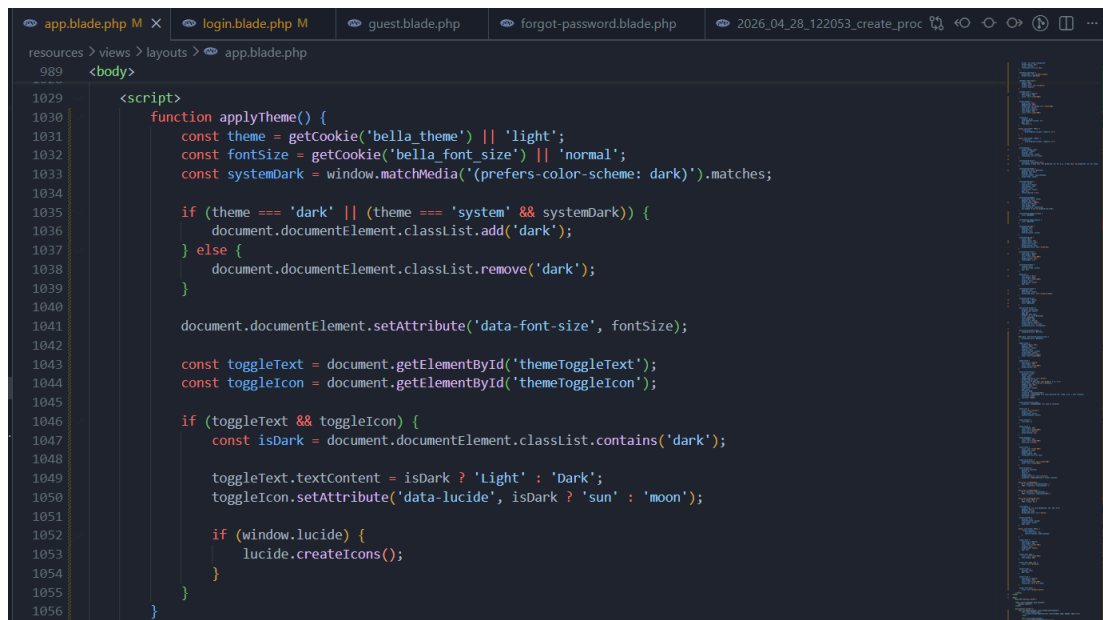
b. Helper Function Cookie



```
resources > views > layouts > app.blade.php
3 <head>
6 <script>
7     function setCookie(name, value, days) {
8         let expires = "";
9
10        if (days) {
11            const date = new Date();
12            date.setTime(date.getTime() + (days * 24 * 60 * 60 * 1000));
13            expires = "; expires=" + date.toUTCString();
14        }
15
16        document.cookie = name + "=" + encodeURIComponent(value) + expires + "; path=/";
17    }
18
19    function getCookie(name) {
20        const nameEQ = name + "=";
21        const cookies = document.cookie.split(';');
22
23        for (let i = 0; i < cookies.length; i++) {
24            let cookie = cookies[i];
25
26            while (cookie.charAt(0) === ' ') {
27                cookie = cookie.substring(1, cookie.length);
28            }
29
30            if (cookie.indexOf(nameEQ) === 0) {
31                return decodeURIComponent(cookie.substring(nameEQ.length, cookie.length));
32            }
33        }
34
35        return null;
36    }
37
38    function deleteCookie(name) {
39        document.cookie = name + '=; Max-Age=-99999999; path=/';
40    }
41
```

Pada file layout yang kamu kirim, helper function setCookie, getCookie, dan deleteCookie sudah dibuat di bagian atas sebelum halaman dimuat.

c. Toggle Dark Mode



```
resources > views > layouts > app.blade.php
989 <body>
1029 <script>
1030     function applyTheme() {
1031         const theme = getCookie('bella theme') || 'light';
1032         const fontSize = getCookie('bella font_size') || 'normal';
1033         const systemDark = window.matchMedia('(prefers-color-scheme: dark)').matches;
1034
1035         if (theme === 'dark' || (theme === 'system' && systemDark)) {
1036             document.documentElement.classList.add('dark');
1037         } else {
1038             document.documentElement.classList.remove('dark');
1039         }
1040
1041         document.documentElement.setAttribute('data-font-size', fontSize);
1042
1043         const toggleText = document.getElementById('themeToggleText');
1044         const toggleIcon = document.getElementById('themeToggleIcon');
1045
1046         if (toggleText && toggleIcon) {
1047             const isDark = document.documentElement.classList.contains('dark');
1048
1049             toggleText.textContent = isDark ? 'Light' : 'Dark';
1050             toggleIcon.setAttribute('data-lucide', isDark ? 'sun' : 'moon');
1051
1052             if (window.lucide) {
1053                 lucide.createIcons();
1054             }
1055         }
1056     }

```

Pada layout juga sudah terdapat logic untuk menambahkan atau menghapus class dark pada HTML sesuai cookie tema.

d. Cookie Diterapkan di Awal Load

```
resources > views > layouts > app.blade.php
3 <head>
6 <script>
41
42 function applyThemeBeforeLoad() {
43     const theme = getCookie('bella_theme') || 'light';
44     const fontSize = getCookie('bella_font_size') || 'normal';
45     const systemDark = window.matchMedia('(prefers-color-scheme: dark)').matches;
46
47     if (theme === 'dark' || (theme === 'system' && systemDark)) {
48         document.documentElement.classList.add('dark');
49     } else {
50         document.documentElement.classList.remove('dark');
51     }
52
53     document.documentElement.setAttribute('data-font-size', fontSize);
54 }
55
56 applyThemeBeforeLoad();
57 </script>
```

Kode ini diletakkan di dalam tag `<head>`, sehingga tema langsung diterapkan sebelum halaman tampil sepenuhnya. Ini mencegah tampilan kedip atau flash saat berpindah mode.

e. Halaman Preferensi

```
> ProductSeeder.php X DatabaseSeeder.php welcome-splash.blade.php navbar.blade.php M preferensi.blade.php U X Bella Admin
resources > views > preferensi.blade.php
79 <div class="preference-wrapper">
87 <div class="preference-card">
94 <form id="preferenceForm">
95     @csrf
96
97     <div class="preference-row">
98         <label for="theme" class="preference-label">Pilih Tema</label>
99         <select name="theme" id="theme" class="preference-select">
100             <option value="light" {{ $theme === 'light' ? 'selected' : '' }}>Light</option>
101             <option value="dark" {{ $theme === 'dark' ? 'selected' : '' }}>Dark</option>
102             <option value="system" {{ $theme === 'system' ? 'selected' : '' }}>System</option>
103         </select>
104     </div>
105
106     <div class="preference-row">
107         <label for="font_size" class="preference-label">Ukuran Font</label>
108         <select name="font_size" id="font_size" class="preference-select">
109             <option value="small" {{ $fontSize === 'small' ? 'selected' : '' }}>Small</option>
110             <option value="normal" {{ $fontSize === 'normal' ? 'selected' : '' }}>Normal</option>
111             <option value="large" {{ $fontSize === 'large' ? 'selected' : '' }}>Large</option>
112         </select>
113     </div>
114
115     <button type="submit" class="btn-save-preference">
116         Simpan Preferensi
117     </button>
118 </form>
119
120 <div id="preferenceAlert" class="preference-alert"></div>
121 </div>
122 </div>
123 @endsection
```

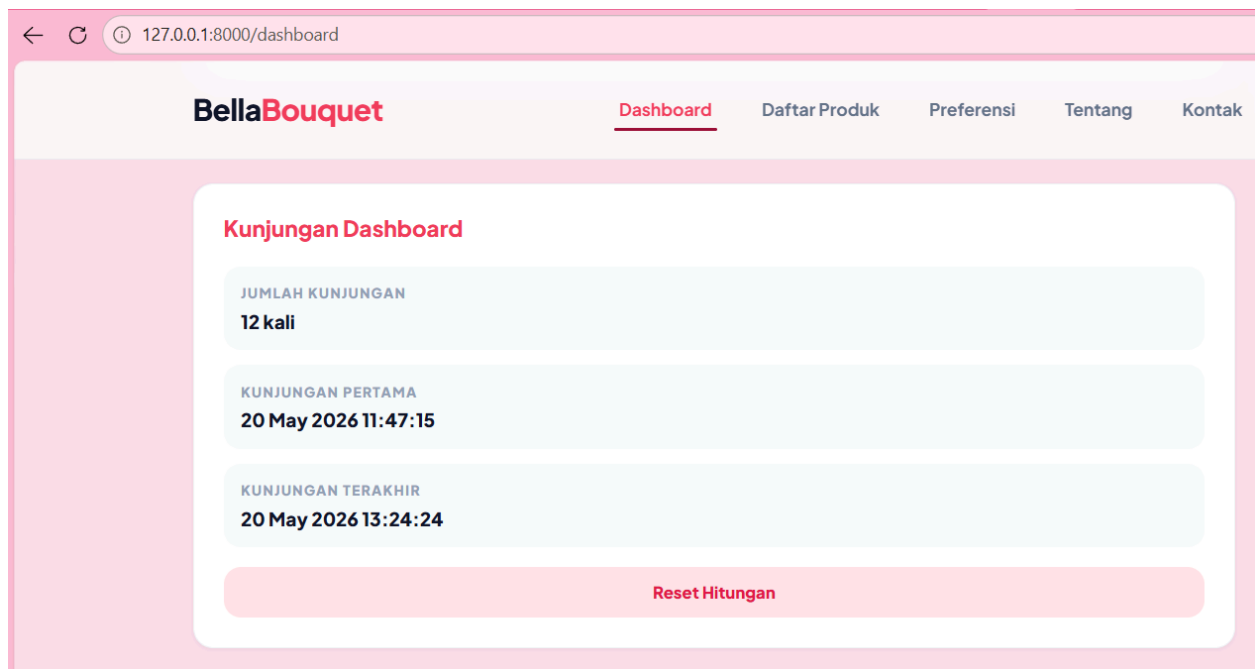
f. Simpan Preferensi via Fetch POST

```
resources > views > preferensi.blade.php
125 @push('scripts')
126 <script>
127     document.getElementById('preferenceForm').addEventListener('submit', async function (e) {
128         e.preventDefault();
129
130         const form = e.target;
131         const formData = new FormData(form);
132         const alertBox = document.getElementById('preferenceAlert');
133
134         try {
135             const response = await fetch("{{ route('preferensi.save') }}", {
136                 method: "POST",
137                 headers: {
138                     "X-CSRF-TOKEN": document.querySelector('meta[name="csrf-token"]').getAttribute('content'),
139                     "Accept": "application/json"
140                 },
141                 body: formData
142             });
143
144             const result = await response.json();
145
146             if (result.success) {
147                 setCookie('bella_theme', formData.get('theme'), 30);
148                 setCookie('bella_font_size', formData.get('font_size'), 30);
149                 applyTheme();
150
151                 alertBox.style.display = 'block';
152                 alertBox.textContent = result.message + ' Tema: ' + result.cookie_baru.theme + ', Font: ' + result.cookie_baru.font_size;
153             }
154             catch (error) {
155                 alertBox.style.display = 'block';
156                 alertBox.textContent = 'Gagal menyimpan preferensi. Coba ulangi lagi.';
157             }
158         });
159     </script>
160 @endpush
```

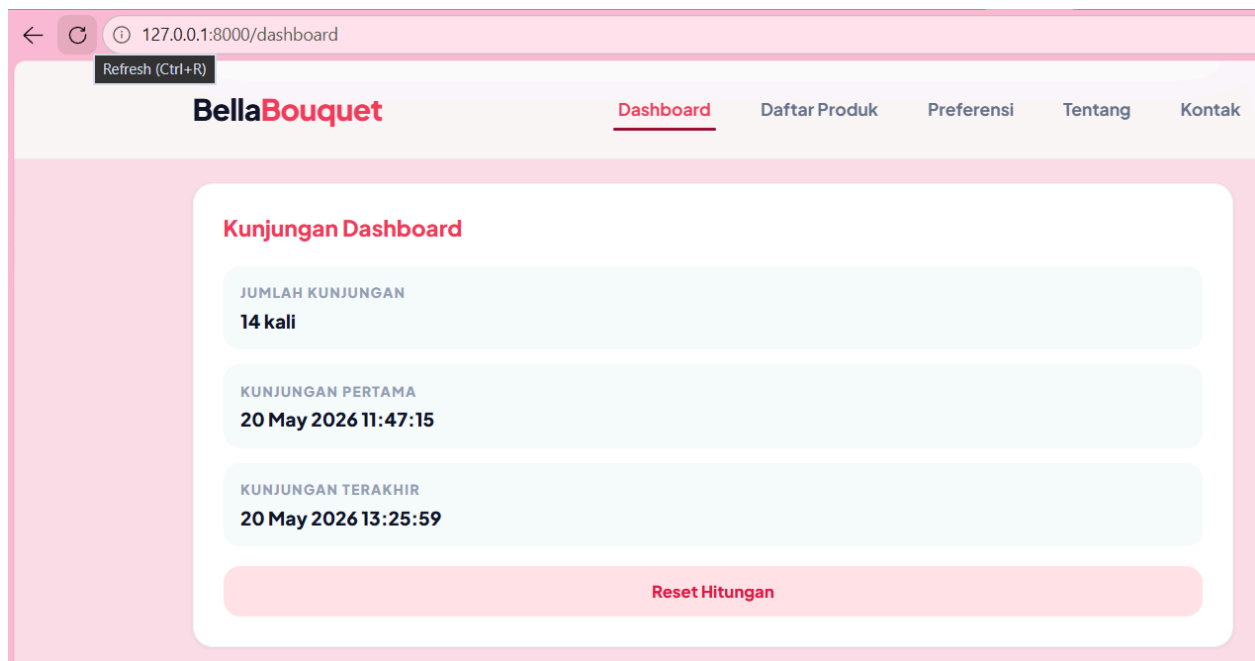
g. Controller Mengembalikan JSON

```
.php U | Bella Admin Management | stat-card.blade.php | ProductController.php M | Controller.php | PreferenceController.php U X
app > Http > Controllers > PreferenceController.php
5 use Illuminate\Http\Request;
6
7 class PreferenceController extends Controller
8 {
9     public function index(Request $request)
10     {
11         $theme = $request->cookie('bella_theme', 'light');
12         $fontSize = $request->cookie('bella_font_size', 'normal');
13
14         return view('preferensi', compact('theme', 'fontSize'));
15     }
16
17     public function save(Request $request)
18     {
19         $request->validate([
20             'theme' => 'required|in:light,dark,system',
21             'font_size' => 'required|in:small,normal,large',
22         ]);
23
24         $oldTheme = $request->cookie('bella_theme', 'belum ada');
25         $oldFontSize = $request->cookie('bella_font_size', 'belum ada');
26
27         $theme = $request->theme;
28         $fontSize = $request->font_size;
29
30         $response = response()->json([
31             'success' => true,
32             'message' => 'Preferensi berhasil disimpan ke cookie.',
33             'cookie_lama' => [
34                 'theme' => $oldTheme,
35                 'font_size' => $oldFontSize,
36             ],
37             'cookie_baru' => [
38                 'theme' => $theme,
39                 'font_size' => $fontSize,
40             ],
41         ]);
42     }
43 }
```

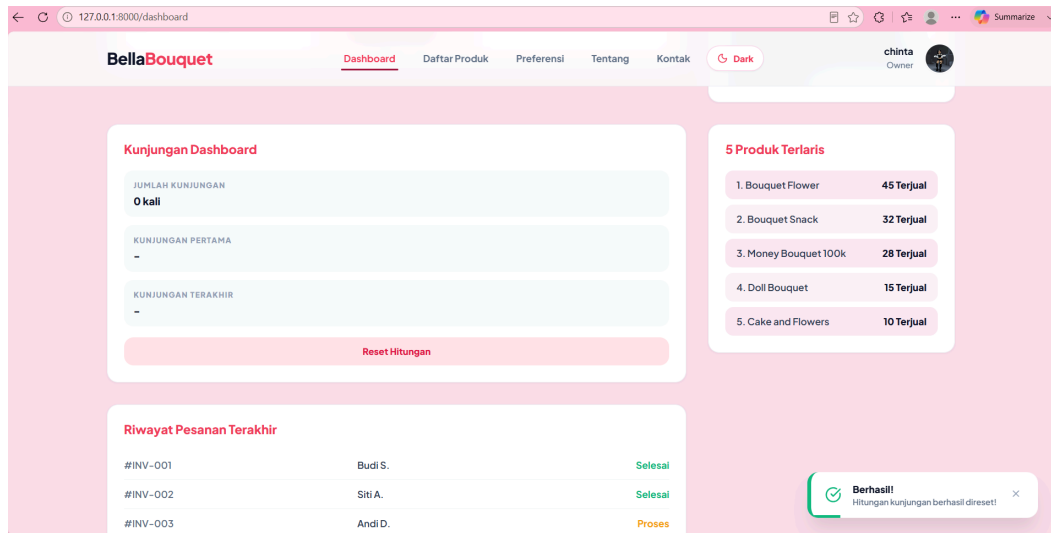
4. Section Hitungan Kunjungan Menggunakan Session Laravel



Screenshot ini menunjukkan jumlah kunjungan, waktu kunjungan pertama, dan waktu kunjungan terakhir.



Screenshot ini membuktikan bahwa session kunjungan berjalan.



Screenshot ini menunjukkan fitur reset session.

Code :

```

EXPLORER
BELLA-BOUQUET
  app
    Http
    Controllers
      PreferenceC... U
      ProductCont... M
      ProfileContr... M
    Middleware
      CekAdmin.php
    Requests
    Models
      Product.php M
      Tag.php
      User.php M
    Providers
    View
    bootstrap
    cache
    app.php
    providers.php
    config
    database
    factories
    migrations
      0001_01_01_00000...
      0001_01_01_00000...
      0001_01_01_00000...
      2026_04_28_12205...
      2026_05_09_06041...

app > Http > Controllers > DashboardController.php
You, 29 seconds ago | 1 author (You)
1 <?php
2
3 namespace App\Http\Controllers;
4
5 use Illuminate\Http\Request;
6 use Illuminate\Support\Facades\Route;
7 use App\Models\Product;
8
9 class DashboardController extends Controller
10 {
11     public function index(Request $request)
12     {
13         $skipIncrement = $request->session()->pull('skip_visit_increment', false);
14
15         if (!$skipIncrement) {
16             $visitCount = $request->session()->get('dashboard_visit_count', 0) + 1;
17
18             if ($request->session()->has('dashboard_first_visit')) {
19                 $request->session()->put('dashboard_first_visit', now()->translatedFormat('d F Y H:i:s'));
20             }
21
22             $request->session()->put('dashboard_visit_count', $visitCount);
23             $request->session()->put('dashboard_last_visit', now()->translatedFormat('d F Y H:i:s'));
24         }
25
26         $visitData = [
27             'count' => $request->session()->get('dashboard_visit_count', 0),
28             'first' => $request->session()->get('dashboard_first_visit', '-'),
29             'last' => $request->session()->get('dashboard_last_visit', '-'),
30         ];
31     }
32 }
  
```

Reset_kunjungan :

```

public function resetKunjungan(Request $request)
{
    $request->session()->forget([
        'dashboard_visit_count',
        'dashboard_first_visit',
        'dashboard_last_visit',
    ]);

    $request->session()->put('skip_visit_increment', true);

    return redirect()->route('dashboard')->with('success', 'Hitungan kunjungan berhasil direset!');
}
  
```

File Route:

```

web.php M X  tailwind.config.js M  profil.blade.php M  ProfileController.php M  2026_05_20_063917_add_profile_fields  ↻ ↺ ↻ ↻ ↻
routes > web.php
10 Route::middleware(['auth'])->group(function () {
19
20 // Dashboard + session kunjungan
21 Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard');
22 Route::post('/dashboard/reset-kunjungan', [DashboardController::class, 'resetKunjungan'])->name('dashboard.resetKunjungan');
23

```

File View:

The screenshot shows a code editor with a file explorer on the left. The 'resources' folder is expanded, showing subfolders like 'css', 'js', 'views', and 'layouts'. The 'views' folder is further expanded, showing 'auth', 'components', and 'layouts'. The 'layouts' folder is selected, and the 'dashboard.blade.php' file is open in the main editor. The code in the editor is a Laravel Blade template for a dashboard. It includes a grid of visit items and a reset button. The code is as follows:

```

resources > views > dashboard.blade.php
293 <div class="dashboard-layout">
384 </div>
386 <div class="widget-card">
387 <h3 class="widget-title">Kunjungan Dashboard</h3>
388
389 <div class="visit-grid">
390 <div class="visit-item">
391 <div class="visit-label">Jumlah Kunjungan</div>
392 <div class="visit-value">{{ $visitData['count'] }} kali</div>
393 </div>
394
395 <div class="visit-item">
396 <div class="visit-label">Kunjungan Pertama</div>
397 <div class="visit-value">{{ $visitData['first'] }}</div>
398 </div>
399
400 <div class="visit-item">
401 <div class="visit-label">Kunjungan Terakhir</div>
402 <div class="visit-value">{{ $visitData['last'] }}</div>
403 </div>
404 </div>
405
406 <form method="POST" action="{{ route('dashboard.resetKunjungan') }}">
407 @csrf
408 <button type="submit" class="btn-reset-session">
409 Reset Hitungan
410 </button>
411 </form>
412 </div>
413

```

Penjelasan

Fitur ini menggunakan session Laravel untuk menyimpan jumlah kunjungan pengguna pada halaman dashboard. Setiap kali halaman dashboard dibuka, jumlah kunjungan bertambah satu. Waktu kunjungan pertama disimpan saat pertama kali pengguna membuka halaman, sedangkan waktu kunjungan terakhir diperbarui setiap kali halaman dikunjungi. Tombol reset digunakan untuk menghapus data session dan mengulang hitungan dari awal.